

Measuring Data Exposure in LLM-Integrated Productivity Tools

Marawan Eldeib

Matrikelnummer: 3764796

University of Stuttgart

Supervisor: Prof. Dr. Marco Aiello

September 17, 2026

Abstract—Large-language-model (LLM)-integrated writing assistants run as browser extensions that silently observe the text a user types or pastes into web-page fields and send it to their servers for analysis. We measure how much of a confidential document such tools transmit by default, even when the user has been told not to share it with any AI tool, using an HTTPS man-in-the-middle proxy and a synthetic memo carrying twelve unique planted identifiers, including a UUID canary. Across five runs each, Grammarly and LanguageTool transmitted essentially the entire document (99.0% and 91.9% of characters) and all twelve secrets, including the canary. Extending the measurement to the host operating system shows the exposure is surface-dependent: merely opening the file in Microsoft Word, with no paste and no submission, streams 96.8% of it to Microsoft’s default-enabled Office AI cloud, and disabling a single Office setting stops it entirely. Content leaves the machine whenever it enters an LLM-integrated surface, independent of where the file is stored; we frame this as a gap between default behaviour and reasonable user expectation rather than an exploit. All test data is synthetic and the methodology is reproducible.

Index Terms—Browser extensions, data exposure, HTTPS interception, LLM-integrated productivity tools, PII leakage

I. INTRODUCTION

Large-language-model (LLM) integrated productivity tools (grammar checkers, paraphrasers, and “AI assistant” extensions) have become a default part of many users’ browsers. They work by observing the content of text fields and sending that content to remote servers for analysis, then returning suggestions. The convenience is real; the privacy cost is largely invisible. A user who has been handed a confidential file and told “do not share this with any AI tool” would not knowingly paste it into a chatbot. But that same user may have a grammar-checking extension enabled in the background, silently watching every field they type or paste into. The question this project asks is deliberately mundane:

How does user data exposure differ across LLM-integrated productivity tools during controlled use?

Concretely: when a confidential document is pasted into an ordinary text field with a writing-assistant extension running by default, how much of that document leaves the machine, and does it include the specifically sensitive parts? Grammar-checking browser extensions are the entry point, but the study spans the integration types these tools take in practice: browser extensions, desktop and editor applications, and system-level services.

This is not a search for a vulnerability. The tools examined here are doing exactly what they were designed to do. Our contribution is a reproducible measurement of this default behaviour and an analysis of how it diverges from users’ expectations. Our contributions are:

- A measurement method (intercept HTTPS traffic, search it for planted secrets including a unique UUID), tested on Grammarly and LanguageTool against a no-extension baseline.
- The same test on the host system, showing the leak depends on the surface a document enters, not on where the file is stored.
- That turning off one Office setting stops the Word leak, and that the tools rely on all-sites text access, not microphone or camera.

II. BACKGROUND AND THREAT MODEL

How these tools work.: A writing-assistant extension registers content scripts that attach to a page’s editable fields, that is, text boxes and rich-text areas (in web terms, `<textarea>` and `contenteditable` elements), on visited pages. When text appears in such a field, the extension transmits it, typically over HTTPS or a WebSocket, to its backend, which runs the analysis and returns suggestions. Because the analysis is server-side, the text itself must leave the user’s machine for the feature to function at all.

Intended use versus incidental exposure.: A user’s text can reach one of these tools in two ways. In intended use, the user knowingly hands over the text: they type into the tool’s own editor, or they click an action such as “paraphrase” or “summarise.” Transmission is then expected and effectively consented to. The privacy question lies in the other case, incidental exposure: the extension is installed and enabled, the default state for millions of users, and it captures text the user entered for an unrelated purpose, with no intent to use the tool and no action beyond typing or pasting. We first confirmed the intended-use mechanism, that server-side analysis makes the text leave the machine at all, then measured the incidental case across a set of tools under identical conditions. This is how the project’s question, how data exposure differs across tools during controlled use, is answered: by comparing tools on the exposure that occurs when the user has not chosen to share.

Threat model.: We model an ordinary, non-adversarial situation:

A user receives a confidential file and is explicitly told not to share it with AI tools. They have a writing-assistant extension installed and enabled, the default state for millions of users. In the course of normal work they paste part of the document into a normal text field (an email reply, a web form, a notes box). Without any further action, the extension transmits the content to its servers.

The adversary here is not a criminal; it is the gap between the tool’s default behaviour and the user’s mental model. The user believes they have withheld the document from AI tools. In fact, merely pasting it into a field an unrelated extension happens to watch has already transmitted it. Our measurement targets precisely this gap: exposure that occurs with no user action beyond pasting. The browser paste is our primary scenario; we later broaden the same question to desktop applications and to the act of merely opening a received file (Section V-J), where in some cases even the paste is unnecessary.

III. RELATED WORK

The privacy risks of browser extensions have been studied for the better part of a decade. Starov and Nikiforakis [30] were among the first to systematically measure the “privacy diffusion” enabled by extensions, showing that many leak identifying information far beyond what their function requires. Weissbacher et al. [33] built Ex-Ray to detect history-leaking extensions, and Aggarwal et al. [1] analysed and detected “spying” extensions that exfiltrate (covertly transmit) browsing behaviour. Chen and Kapravelos [8] scaled dynamic taint tracking to 181,683 extensions, flagging thousands that leak privacy-sensitive information. These works established that extension data collection is widespread and frequently undisclosed. They also introduced the complementary detection methods the field still relies on: differential string search, content-agnostic network analysis, and taint tracking. Static data-flow analysis (e.g. DoubleX [18]) later added scale on the code side. This line of work rests on a longer study of the extension security model itself: Carlini et al. [6] evaluated Chrome’s permission and isolation architecture and showed that over-broad host permissions undercut its guarantees, and Pantelaios et al. [26] showed that benign extensions can turn malicious through later updates, which is why a runtime, network-level measurement of what a tool actually transmits is more reliable than trusting a single reviewed snapshot.

More recent large-scale studies have moved from browsing metadata to page content. Nayak et al. [25] experimentally analysed sensitive-data access by extensions, including access to credentials on login pages, across the Chrome Web Store. Xie et al. [34] built Arcanum, a dynamic taint-tracking system, and found that thousands of extensions, used by an estimated 144 million people, collect sensitive on-page content such as email bodies, social-media activity, and banking information, often without accurate disclosure. Closer to our own design,

Bui et al. [5] intercepted the network traffic of roughly 47,000 extensions and compared the observed outbound data flows against each extension’s stated privacy practices, finding hundreds whose transmissions contradicted their disclosures. From the tracking side, Dambra et al. [14] measured web tracking from the user’s perspective, building on the browser-instrumentation measurement paradigm established by OpenWPM [17].

The most behaviourally similar precedent is “read-before-submit” exfiltration: Senol et al. [28] showed that email addresses and even passwords are sent to third parties as the user types, before any form is submitted; Munir et al. [23] recently generalised this, finding widespread third-party keystroke event-listener interception across popular sites, and Starov et al. [31] earlier quantified personally identifiable information (PII) leaking from website contact forms. We extend that “transmitted before the user acts” framing from trackers to writing-assistant extensions. Methodologically, our detection step, searching intercepted flows for known sensitive strings, follows a mature network-level PII-measurement lineage rooted in dynamic taint tracking (TaintDroid [16]) and network-level flow analysis over decrypted mobile traffic (ReCon [27]). Because outbound payloads can be encoded or obfuscated, we adopt unique planted canaries in the spirit of the black-box differential leak detection introduced by AGRIGENTO [11], so that any transmitted fragment is unambiguously attributable to our input.

Closest to the present work, Vekaria et al. [32] audited nine generative-AI browser assistants and, using traffic decryption, showed that several forward full webpage content, including medical and banking details, to their own servers and to third-party trackers. Our study is narrower and complementary: rather than a broad crawl, we perform a deep, controlled, and fully reproducible measurement of a specific, common threat model (a confidential document pasted into a field with a grammar checker running by default), using a planted-canary methodology that makes each leak individually and unambiguously attributable. Where prior large-scale work answers “how many extensions leak,” we answer “exactly how much of a specific confidential document leaves, and which sensitive tokens, for two representative tools, with what reproducibility.”

Finally, why the transmitted content matters is reinforced by a parallel line of work on the privacy risks of large language models, which increasingly sit behind these assistants. Carlini et al. [7] showed that language models memorise and can regurgitate verbatim personal data from their training set, and Nasr et al. [24] extended such extraction to aligned, production models, recovering training examples from deployed systems such as ChatGPT; Lukas et al. [19] quantified PII leakage from language models and showed that scrubbing and differential privacy leave residual exposure. Staab et al. [29] showed that models can infer sensitive attributes (location, age, income) from ordinary prose; and Mireshghallah et al. [22] showed that LLM-integrated applications routinely disclose information to recipients that are inappropriate for the context. Text sent to a writing assistant is therefore not inert once it leaves

the machine, which is precisely what makes quantifying how much of it leaves, and to whom, worthwhile. This gap between what users believe about their exposure and what actually leaves their devices has itself become a subject of study: recent work frames a “privacy paradox” between user perceptions and the reality of PII leakage in LLM-integrated settings [9], motivating the concrete, quantified measurement we provide here.

Once the text leaves the machine, it can be kept and reused by others. Threat-intelligence reporting has found deployed AI services and unauthorised “model-reseller” proxies silently storing and passing on users’ inputs, including names, contact details, and corporate data, without their knowledge [2]. Industry analyses report the same for the tools we study: a Unit 42 investigation found “productivity” browser extensions that read users’ email as they write it and capture page content [4], and enterprise-security reporting notes that such extensions read page content largely invisibly to standard data-loss-prevention tools, with much of the data coming from personal, unmonitored accounts [10, 13].

Research gap.: Prior work either counts how many extensions leak at scale or audits AI assistants broadly; what is missing is a controlled, canary-proven, per-secret measurement of how much of a single confidential document leaves the machine, across both the browser extensions and the everyday host applications (Office, desktop clients, cloud-AI viewers) a real user actually encounters. This paper fills that gap.

IV. METHODOLOGY

A. Study design and conditions

We measure three conditions: two writing-assistant extensions and a control. **Grammarly** is the always-on grammar and style checker that motivated this project. **LanguageTool** is a grammar checker from a different company (LanguageTooler GmbH, based in Germany) that needs no account, which makes it a clean second tool for checking whether the same behaviour repeats. The **baseline** is the same browser and steps with no extension installed, used to subtract the browser’s ordinary background traffic. Testing two independent tools rather than one gives stronger evidence: if both behave the same way, the result is not a quirk of a single product.

B. Test document and planted secrets

The input is a synthetic “confidential memo” (2,065 characters, 245 words) written to look like an internal compliance document and explicitly marked “DO NOT FORWARD.” Embedded in its prose are **twelve unique, fictional identifiers**: names, an email address, a phone number, project and reference codes, and a UUID canary, CANARY-BC267061-67DC-485B-8E51-6F5494765CAB. None of these strings exists anywhere on the public internet, so any occurrence in captured network traffic is unambiguously attributable to our input. The canary in particular gives a single, unforgeable yes/no signal: if it appears in outbound traffic, the confidential payload provably reached the tool’s servers. Table I lists the twelve identifiers and their categories.

TABLE I: The twelve planted identifiers embedded in the synthetic memo.

Category	Item
Name (direct identifier)	Helena Voss
Name (direct identifier)	Theodora Baumgartner-Klein
Contact detail	Email address
Contact detail	Phone number
Official / ID number	Employee ID
Official / ID number	Tax registration number
Financial identifier	Q1 expenditure reference
Financial identifier	Reserve fund code
Financial identifier	Budget approval code
Legal / contractual reference	Contract number
Confidential project ID	Project codename
Canary proof-token	Unique UUID leak marker

C. Interception setup

For the browser phase, all captures were performed on an isolated Kali Linux virtual machine so that the man-in-the-middle configuration never touched real browsing. We used mitmproxy [12] (version 12.2.3) as an intercepting HTTPS/WebSocket proxy on 127.0.0.1:8080. Each tool ran in its own clean Firefox profile with the proxy configured and the mitmproxy certificate authority trusted only within that profile, so decryption was scoped to the experiment. Before any data collection, interception was verified per profile (a known HTTPS site must load without a certificate warning and appear in the proxy); this guards against the most dangerous failure mode in such studies, in which a broken interception silently looks identical to a genuine zero-exposure result.

The document was pasted into a bare local HTML page containing a single text field and nothing else (no third-party scripts, analytics, or other domains) served over http://localhost so that extension content scripts activate exactly as they would on a real site, but with zero background noise to filter out.

D. Capture procedure

For each run, the document was loaded into the clipboard and pasted manually into the focused field, after which the run recorded for sixty seconds. This detail matters: a scripted or synthetic paste fills the field visually but does not fire the DOM input event the extensions listen for, so the tool never ingests the text and nothing is transmitted, a misleading “clean” zero. A real, manual paste does fire that event, so in our runs the tools genuinely received the document. Each tool was captured for **five runs** and the baseline for **three**, following an identical, locked protocol so that between-run variance measures reproducibility rather than procedural drift.

E. Host and system-level phase

Several desktop tools (Microsoft Office, the DeepL desktop client, Edge/Copilot) are not available on Linux, so the host phase was run on a Windows 11 laptop using the same interception principle. mitmproxy ran the same scoring add-on, and both the WinINET and WinHTTP system proxies were

Measurement pipeline

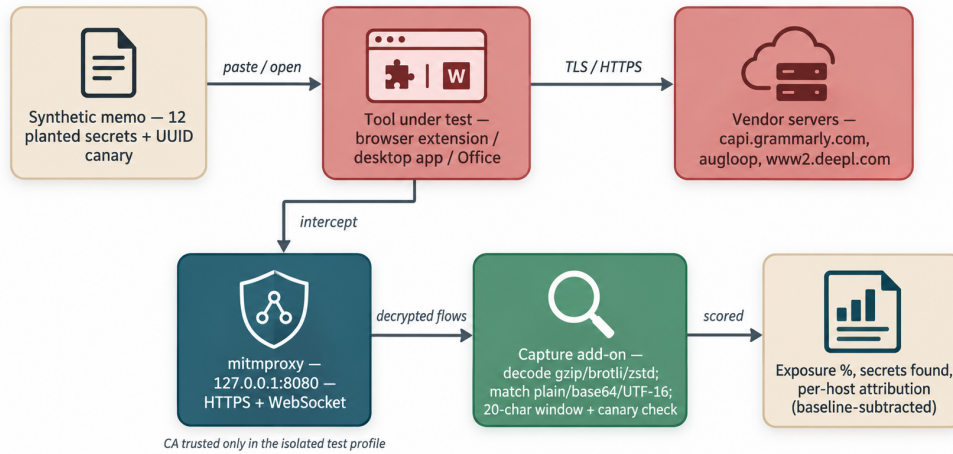


Fig. 1: Overview of the measurement pipeline. The synthetic memo (with twelve planted secrets and a UUID canary) enters the tool under test, whose traffic is intercepted by mitmproxy on 127.0.0.1:8080; a capture add-on decodes and matches the outbound flows to produce a baseline-subtracted exposure figure and per-host attribution.

Browser phase — isolated Kali Linux VM

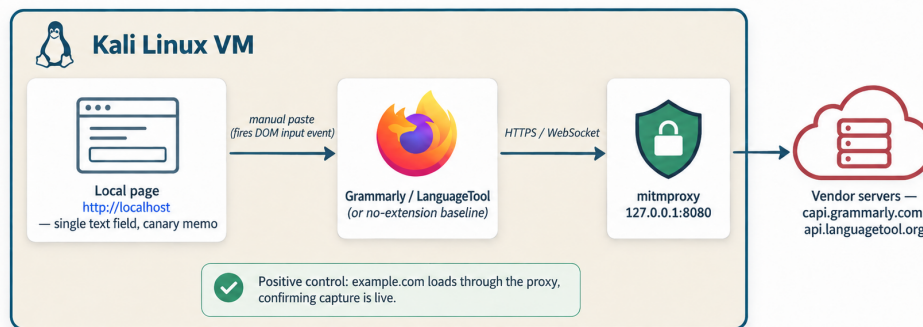


Fig. 2: Browser phase. On an isolated Kali Linux VM, the canary memo is pasted into a bare local page; the extension reads the field and transmits over HTTPS/WebSocket through mitmproxy to the vendor. A positive control (example.com) confirms interception is live.

pointed at it. WinINET carries browser and interactive-app traffic; WinHTTP carries native and background apps. Office and the Grammarly desktop client use WinHTTP and would bypass a WinINET-only (browser) proxy, so covering both is what makes the host capture complete. Each condition used the same synthetic memo and canary. Because a real machine runs several assistants at once, every captured flow was attributed to its destination host, so a leak is credited to the specific tool responsible rather than to whatever else happened to be running.

F. Metrics

A mitmproxy add-on scored every decrypted outbound request, response, and WebSocket frame; the raw captures were also archived so they can be re-analysed offline. We report, in order of importance:

Planted-secret count (primary).

How many of the twelve identifiers reached the servers, verified individually. The canary makes this unambiguous.

Exposure %.

The fraction of the document's characters whose 20-

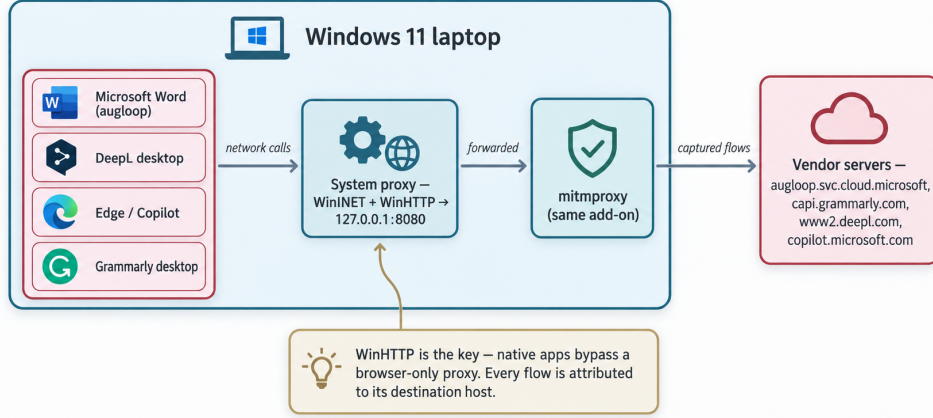


Fig. 3: Host phase (Windows 11). Native applications and Office reach the network through the system proxy (WinINET + WinHTTP), which forwards to mitmproxy. WinHTTP is required because native clients bypass a browser-only proxy; every flow is attributed to its destination host.

character sliding window appears anywhere in outbound traffic, after subtracting hosts seen in the baseline:

$$\text{Exposure (\%)} = \frac{|\{i : w_{20}(i) \in \mathcal{T}\}|}{N} \times 100, \quad (1)$$

where $w_{20}(i)$ is the 20-character window starting at character position i , N is the document length in characters, and $\mathcal{T}$ is the baseline-subtracted outbound traffic matched over all decoded variants. A 20-character window is long enough to avoid false matches from common words and short enough to catch fragmented sends; a per-host concatenated-transcript pass (all of a host's traffic joined into one stream) additionally catches content split across multiple frames. Matching is performed over plain, URL-decoded, JSON-unescaped, HTML-entity, base64-decoded, and UTF-16 variants so that encoded transmission is still detected, and compressed bodies (gzip, brotli, zstd) are decoded before matching.

Reproducibility.

The standard deviation of exposure % across a tool's runs, in percentage points (pp), labelled High (< 3 pp), Medium (3–10 pp), or Low (> 10 pp).

Traffic visibility.

Reported as two separate numbers: the share of intercepted events that used HTTPS, and the count of TLS handshakes that could not be intercepted (for example due to certificate pinning¹), never combined into a single figure, because a non-zero failure count means the exposure result is a lower bound (traffic we cannot decrypt could only add to what was sent, never subtract).

¹Certificate pinning is when an application accepts only its own expected TLS certificate, so an intercepting proxy cannot decrypt that connection; the TLS handshake is the initial exchange that sets up the encrypted channel.

For conditions with run-to-run variation we also report a 95% confidence interval on the mean exposure (a range the true average is likely to fall in); where two conditions' intervals do not overlap, we treat that as informal evidence of a real difference. For the browser tools every run was identical (zero variance), so the interval collapses to the reported value.

Code and data availability.: The capture and analysis code, the synthetic memo, and the planted-secret list are available at <https://github.com/MarawanEldeib/privacy-analysis-research>.

V. RESULTS

A. Headline: both tools transmit the whole document and every secret

Table II and Figure 5 summarise the findings. Both extensions transmitted essentially the entire document and **all twelve planted secrets, including the canary**, to their servers. Grammarly transmitted a median **99.0%** of the document's characters; LanguageTool transmitted **91.9%**. The no-extension baseline transmitted **0.0%** and none of the secrets. All transmission used HTTPS, and there were zero un-interceptable TLS handshakes, so the reported figures are complete, not lower bounds.

B. Information types transmitted

Grouping the twelve planted secrets by type shows that the leak is not confined to any single class of data. Both tools transmitted every information type present in the document (personal names, contact details, official identification numbers, financial codes, a legal contract reference, the confidential project codename, and the unique canary), while the no-extension baseline transmitted none of them (Figure 6). Structured identifiers and regulated personal data are therefore

From captured traffic to an exposure number

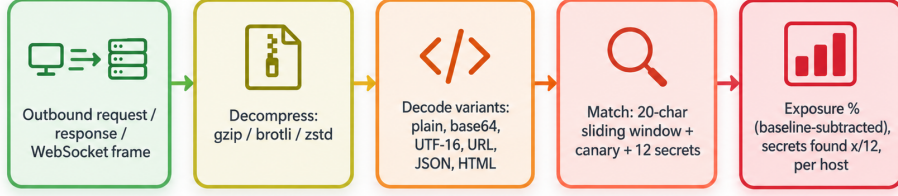


Fig. 4: From captured traffic to an exposure number: each outbound flow is decompressed, decoded across encodings, and matched with a 20-character sliding window plus the canary and twelve secrets, yielding a baseline-subtracted exposure percentage and per-host attribution.

TABLE II: Default-configuration exposure per condition. Exposure is the median across runs; secrets are the number of the twelve planted identifiers detected in outbound traffic. “TLS fail” is the count of handshakes that could not be intercepted.

Condition	Runs	Exposure	Std. dev.	Secrets	Canary?	HTTPS	TLS fail
Grammarly	5	99.0%	0.0 pp	12/12	yes	100%	0
LanguageTool	5	91.9%	0.0 pp	12/12	yes	100%	0
Baseline	3	0.0%	0.0 pp	0/12	no	n/a	0

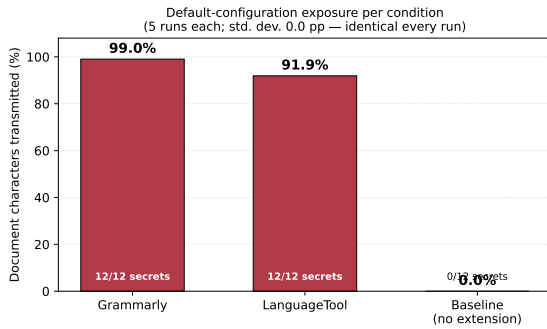


Fig. 5: Share of the confidential document transmitted to each tool’s servers by default. Both grammar checkers transmit essentially the whole document and all twelve planted secrets; the no-extension baseline transmits nothing.

exfiltrated just as readily as free-text prose; nothing about being a formatted ID number affords any protection.

C. Beyond the planted secrets: a transmitted credential

Grammarly’s outbound WebSocket carries more than the user’s document: travelling alongside it is a **JSON Web Token** (an authentication credential, recognisable from its `eyJ...` encoding of a `{"kid":...}` header) together with opaque per-session client identifiers. The same channel that exfiltrates

the confidential text therefore also exposes the tool’s own session credential. LanguageTool’s payloads contain no such artefact. We surfaced this by running an automatic PII detector (Microsoft Presidio) over the decrypted bodies and manually discarding its false positives (document dates, quarter labels, financial figures). Because it is a manual-review result over the existing captures, we report it as an indication rather than a quantified result. The consequence widens the attack surface beyond privacy: exposed session tokens and API keys are exactly what large-scale credential-harvesting operations mine from client-side code and application traffic [2]. A writing assistant that emits its own session credential alongside the user’s text is therefore leaking a directly abusable secret, not just personal data.

D. Exact reproducibility

Both tools produced identical exposure on every run (standard deviation 0.0 percentage points), so we report this as a deterministic observation rather than a sampling estimate: the 95% confidence intervals collapse to points (Grammarly [99.0, 99.0], LanguageTool [91.9, 91.9]) and carry no significance claim. Practically, pasting the document causes the same near-complete transmission every time, consistent with the upload being a fixed part of the tools’ normal operation rather than an occasional event.

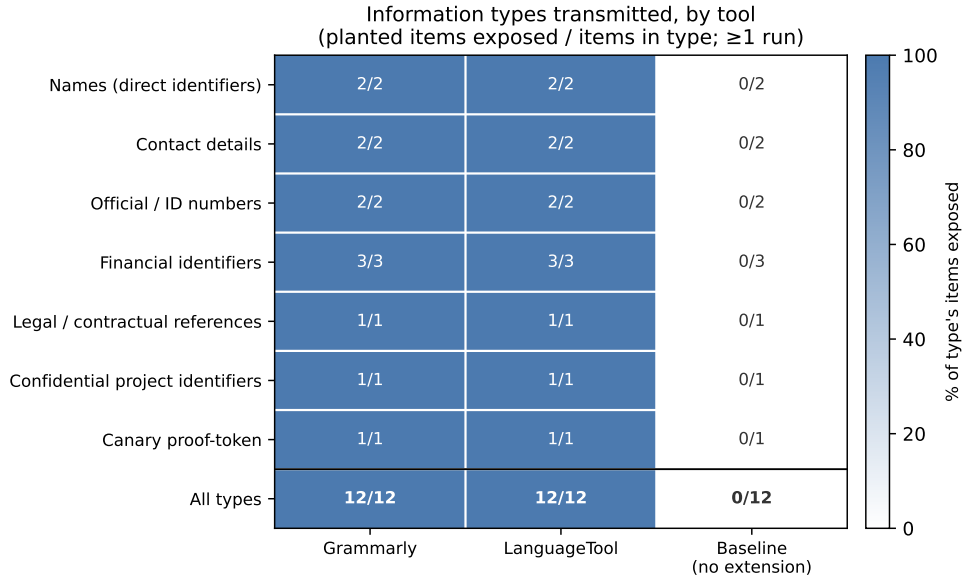


Fig. 6: Information types transmitted to each tool’s servers, grouped from the twelve planted secrets (planted items exposed / items in type, over ≥ 1 run). Both writing assistants expose every type; the no-extension baseline exposes none.

E. What was transmitted, and how

Grammarly streamed the document to its backend over a WebSocket connection (a median of roughly 6 kB of outbound payload per run across about two tool-related hosts). LanguageTool sent it as an HTTPS POST to `api.languagetool.org/v2/check` (roughly 5.6 kB to a single host). In both cases the twelve planted identifiers, including the canary, were recovered from the decrypted outbound traffic. The ~ 8 percentage-point gap between the two tools reflects how much surrounding boilerplate each includes rather than any difference in whether the sensitive content leaves: both transmitted all of the sensitive tokens.

Viewed over time (Figure 7), the transmission appears as a single outbound burst at the moment the document is pasted, after which the connection falls quiet. The no-extension baseline is instructive: it generates more total traffic than either tool run, megabytes of browser and OS background activity, yet none of it carries document content. This is the same reason a baseline condition is required at all: it isolates the environment’s own chatter so that document-bearing traffic can be attributed unambiguously to the installed tool rather than to the browser or operating system.

F. Independent corroboration without a proxy

A natural objection to any man-in-the-middle measurement is whether the proxy itself induced or altered the transmission. We addressed this with an independent capture that removed the proxy entirely: Grammarly’s browser profile was switched to a direct connection, no interception of any kind, and the same document pasted while a packet capture (`tcpdump`) recorded the machine’s traffic, with encrypted-SNI disabled so that destination host names

remained visible. The capture showed the browser resolving `assets.grammarly.com` and uploading several kilobytes directly to Grammarly-owned infrastructure, an AWS EC2 backend (`ec2-44-213-83-95.compute-1.amazonaws.com`) and three CloudFront edges (`*.fra60.r.cloudfront.net`), within the paste window. The payload is opaque in this mode (the browser build does not export TLS session keys, so direct traffic cannot be decrypted without the proxy), but the destination and timing confirm that the upload occurs with nothing intercepting it. The proxy is therefore an observation instrument, not the cause of the transmission.

G. Two exposure classes

The excluded-tool investigation (Section VII) surfaced a distinction worth stating as a result. Writing tools fall into two behavioural classes under this threat model: automatic / background transmitters, which send the document with no user action beyond pasting (Grammarly and LanguageTool), and on-demand transmitters, which send only text the user actively submits (e.g. a paraphraser invoked by selecting text and clicking “rewrite”). Under the study’s exact scenario (an extension running in the background while the user merely pastes), the automatic class leaks and the on-demand class does not. Grammar checkers trend toward the automatic class; paraphrasers toward the on-demand class. This is a meaningful distinction for users and policy, not a measurement artefact.

H. Background behaviour when idle

To separate content transmission from mere background activity, we ran a twenty-minute capture with the Grammarly extension loaded on the test page and no user action at all, no

Traffic over time — the paste spike vs. background
(representative run; blue = carried our document, grey = background)

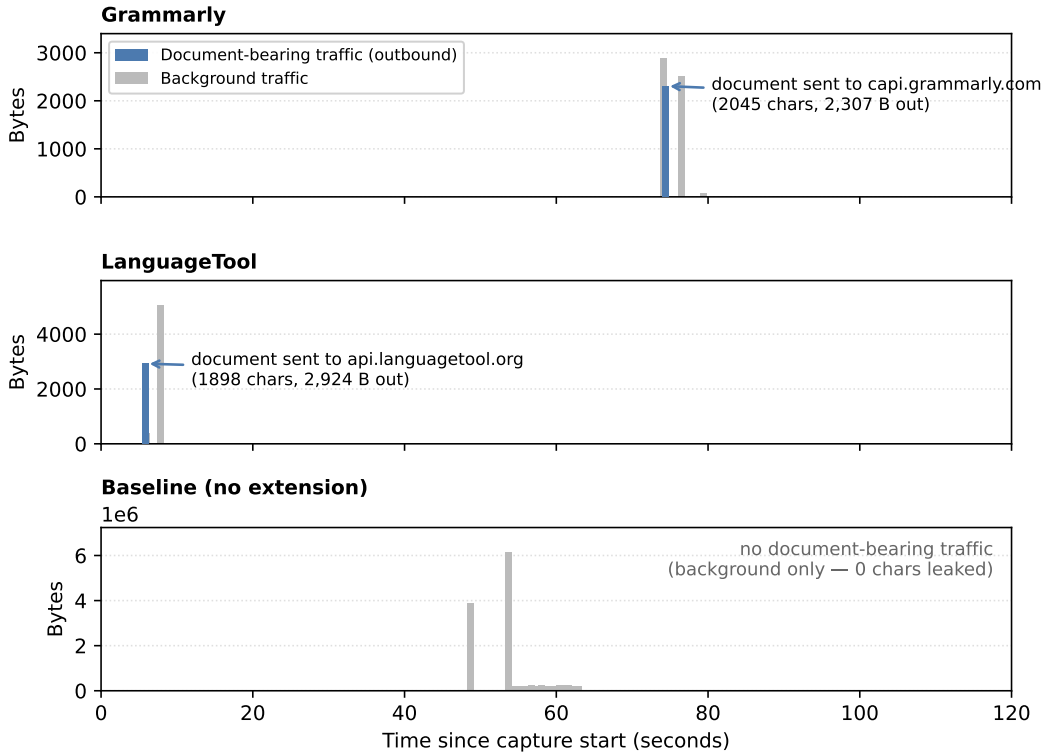


Fig. 7: Traffic over time for a representative run. Both writing assistants transmit the document in a single outbound burst at the moment of paste (blue); the no-extension baseline produces only background traffic (grey) and never transmits document content. Paste timing differs between runs because it was triggered manually; the salient feature is the shape (a single document-bearing spike vs. none), not the absolute time.

paste, no typing, no clicks. The capture path was confirmed live by the presence of the browser’s own background traffic (e.g. `ads.mozilla.org`) as a positive control, without which a quiet result could not be distinguished from a broken capture. Grammarly was not dormant: twenty-three of twenty-seven captured events reached five Grammarly backend hosts: `auth.grammarly.com`, `config.extension.grammarly.com`, `gateway.grammarly.com`, `capi.grammarly.com`, and a logging endpoint `f-log-extension.grammarly.io` (POST /logv2), consistent with authentication, configuration polling, session upkeep, and telemetry. However, **none of this idle traffic carried document content or any planted secret** (0% exposure, 0 of 12). The extension thus maintains a continuous background relationship with the vendor simply by being installed and loaded, while the document itself leaves only once the user supplies text. Both facts matter for the threat model: the tool is always connected, but content exposure is input-triggered.

I. Delivery channel: browser extension vs. desktop client

A preliminary desktop probe reinforces that the exposure is a property of the browser-extension delivery rather than of grammar checking itself. LanguageTool’s official standalone desktop application (version 6.6, obtained from the vendor’s own site) was run through the identical interception setup and given the same document: it performed the check entirely on-device and transmitted **nothing**, 0% of the document and 0 of 12 secrets, whereas the same vendor’s browser extension transmitted 91.9% and all twelve secrets to `api.languagetool.org`. The desktop client exposes no option to use a remote server, so the on-device path is not merely available but the only one. This is a single-run probe on one desktop client, reported as an indication rather than a main result; the fuller desktop and system-level study follows in Section V-J.

J. Desktop and system-level exposure: an action spectrum

To test whether the browser-extension finding generalises, we repeated the measurement on the host operating system (Windows 11) against desktop applications and the everyday

act of opening a received file. The same synthetic document and planted secrets were used, captured with the same mitmproxy add-on; interception on the host required trusting the proxy CA in the Windows store and routing traffic through a system proxy enabled only during capture. The results arrange themselves along a spectrum of user action (Figure 8).

Figure 8 summarises every condition; the prose below highlights only the findings that change the interpretation, naming each receiving host inline.

Active submission leaks, as in the browser phase.:

The **DeepL** desktop client, given the document by paste, transmitted essentially all of it (99.6% mean over three runs) and every secret to `www2.deepl.com`; the whole input was uploaded even though the free tier only translates the first 1500 characters. Invoking **Edge’s Copilot** to summarise the planted PDF transmitted **76.9%** of the file and all twelve secrets to `copilot.microsoft.com` (three runs, identical). The coverage is lower than the paste-based tools not because less leaked but because the source was a PDF: Edge transmits its text extraction of the file, which drops layout and formatting characters, yet every one of the twelve secrets, canary included, was sent. One run stands out: when asked instead for “interesting facts,” Copilot’s visible reply stated it would answer without exposing the confidential details, yet the capture shows the full document, canary included, had already been sent to Microsoft. A model declining to display a secret is therefore not evidence that the secret was not transmitted; the privacy boundary is the network, not the chat reply.

Merely opening a document in Word leaks it.:

Opening the same document as a `.docx`, with no paste and no submission, streamed **96.8%** of it and every secret to `augloop.svc.cloud.microsoft` (Microsoft’s “Augmentation Loop,” the cloud backend for Office’s Editor/Copilot services, via posts to an `OfficeCopilotOrchestrationWorkflow`), reproducibly across three runs. These services are on by default in Microsoft 365 and Microsoft documents that its “connected experiences that analyse content” process document text in the cloud, are enabled by default, and can be disabled through an Office privacy setting, with an EU Data Boundary and GDPR commitments for the processing [21]. The exfiltration therefore came from the default productivity suite itself, not an add-in.

Disabling one setting stops it (causation and mitigation).:

Turning off the single Office setting “experiences that analyse your content” and reopening the same file reduced augloop’s transmission to **0%** (0/12), with the socket still opening but carrying no document content. This both establishes that the Office AI feature, not the act of opening a file, is the cause, and gives users a concrete one-switch mitigation. It removes Microsoft’s own channel only, however: a third-party assistant hooking the same window is unaffected, as the Protected View result next shows.

Protected View does not stop a third-party assistant.:

When the file carried a mark-of-the-web and opened in Word’s sandboxed Protected View (the yellow “Enable Editing” ban-

ner still showing, no user action), Microsoft’s own augloop stayed silent, Protected View contained it, but the **Grammarly** desktop client, hooking the text at the accessibility layer, still transmitted **94.2%** of the document and every secret to `capi.grammarly.com`. Protected View thus sandboxes macros and even Microsoft’s own AI, but not an independent assistant reading the same window.

A file merely stored on a USB or cloud drive is not read by the tools.: A distinct question is whether a confidential file leaks simply by being stored on a removable drive or a synced cloud folder while the writing assistants run in the background. It does not. Inserting the USB without opening the file (only the Windows indexer and shell probed volume metadata, per Process Monitor) and dropping the file into iCloud Drive to sync each transmitted **no** document content to the writing tools’ servers (0/12), even though Grammarly and the other clients were running throughout: the file never entered a surface they watch. iCloud’s own content channels were reachable but carried no cleartext: the file syncs to Apple encrypted in transit and, under Advanced Data Protection, end-to-end [3], so it reaches Apple’s cloud but is never exposed to the LLM writing tools. The same held for opening the file in a viewer with no cloud-AI integration (Notepad, Adobe Acrobat, or Chrome’s PDF viewer), and for opening the PDF in Edge without invoking Copilot (0/12; Edge-passive 1.8% of incidental boilerplate). Consumer antivirus (**Avast**) is a separate class rather than an LLM tool: it uploaded an unknown executable to its cloud (CyberCapture) plus reputation and telemetry, but not the document’s contents. Per-host attribution was essential throughout: Grammarly, always running as a background actor, occasionally re-scanned the file and had to be attributed away from the tool under test (e.g. in the antivirus run) rather than mis-credited.

Interpretation.: Exposure is surface-dependent, not storage-dependent: a file on a USB stick or in a synced cloud folder is not read by the writing tools until its text enters a surface they watch, so passively receiving or opening a file transmits nothing unless that surface has a cloud-AI integration watching the content. Content leaves the machine whenever it enters such a surface, whether the user pastes into a writing assistant, invokes an assistant, or merely opens the file in a cloud-AI-enabled editor.

K. Permissions: the channel is text, not the microphone

A natural question is whether these tools reach beyond text into device sensors. Auditing the Windows capability store (mic, camera, location, contacts, full-disk access, and the rest) and each extension’s declared permissions showed that none of the writing or translation tools (Grammarly, LanguageTool, DeepL, QuillBot) holds microphone, camera, or broad-filesystem access: the microphone and camera belong only to communication, voice, and meeting apps. What the writing tools do hold is the browser’s “read and change all your data on all websites” host permission, which is precisely the capability that lets a content script read every editable field. The exposure channel is therefore the text a user types

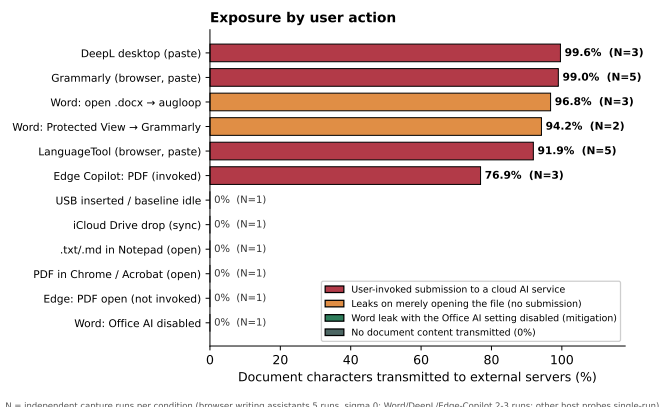


Fig. 8: Document characters transmitted to external servers, by user action. Submitting to a cloud AI tool (DeepL, Grammarly, LanguageTool, invoked Edge Copilot) or merely opening the document in Microsoft Word leaks nearly all of it and every planted secret; disabling one Office setting drops the Word leak to 0%, and passive receipt (USB, iCloud sync) or non-AI viewers (Notepad, Acrobat, browser PDF) transmit no document content. Colour encodes the interaction class; N is the number of independent runs (browser writing assistants 5, standard deviation 0; Word/DeepL/Edge-Copilot 2–3; other host probes single-run), so equal bar lengths should not be read as equal statistical weight.

or pastes, granted through an all-sites content permission, not audio or video surveillance. Grammarly is instructive: its declared extension permissions look modest (“read browsing history,” “display notifications”), yet it transmits 99% of the document, so declared permissions understate actual data access. Consistent with the passive-leak results, the always-on writing clients (Grammarly, DeepL) are configured to launch at start-up, so they are already watching before the user does anything.

VI. DISCUSSION

The clearest single result is the canary, not the percentage. A unique token that existed only inside a document marked “DO NOT FORWARD” was found, seconds after an ordinary paste, in traffic bound for two companies’ servers, placed there by software the user installed once and forgot. Nothing here is a bug or an exploit: server-side analysis requires the text to be uploaded, and both vendors document that they process user text. The finding is the gap between that design and what a user, told “don’t share this with AI,” would expect when they paste a paragraph into an unrelated field.

This distance matters because the exposure is (a) invisible: there is no prompt, no click, no confirmation; (b) complete: essentially the whole document, not a fragment; and (c) deterministic: it happens every time, reliably. For an individual handling regulated data (health, legal, financial) the implication is direct: default-configuration writing assistants are incompatible with a “do not share” instruction unless explicitly

disabled, and disabling them is not the default. The two-class distinction gives users a practical lever: automatic grammar checkers are the ones that leak passively, whereas on-demand paraphraser only transmit what is actively submitted.

Regulatory implications (EU/GDPR and Germany):

Because the memo contains personal data (names and an employee identifier), the behaviour we measure engages the EU General Data Protection Regulation (GDPR, Regulation (EU) 2016/679). A silent, whole-document upload sits in tension with several of its principles: lawfulness and transparency (Art. 5(1)(a), 6), as the transfer is neither disclosed to the user nor obviously grounded in a legal basis; data minimisation (Art. 5(1)(c)), as the entire document is sent rather than the text under review; and the information duties of Arts. 13–14, as the user is never told the text leaves the device. Where the receiving servers sit outside the EU, the transfer additionally engages the Chapter V safeguards (Arts. 44–49). The gap is sharpest in an organisational setting: an employer whose staff paste client data into a consumer writing assistant with no Art. 28 processor agreement is the realistic non-compliance scenario, and in Germany the analysis is stricter still, layering the Federal Data Protection Act (BDSG) and works-council co-determination over tools that can observe employee input. This does not render every use unlawful: Microsoft, for one, documents an EU Data Boundary and a processor agreement for Office’s cloud processing [21], so the same feature can be deployed compliantly. The distinction our results draw is thus practical: vendors that offer a lawful-processing path versus default-on consumer tools that offer none. This makes the invisibility of the transfer, not merely its occurrence, the core data-protection concern. We offer no legal determination; establishing a violation is fact-specific and beyond a measurement study.

Broader impact.: The concern is not only that a document is copied once, but what happens to it afterwards. Text captured by default feeds a vendor’s own processing and, potentially, model training, and once it leaves the machine it can be retained, combined with other data, and passed to parties the user never chose, a pattern users rarely notice because it is authorised deep in terms of service almost no one reads [2]. As these tools gain broader access and more autonomy, the same silent collection scales from individuals to the organisations whose confidential data flows through them. The risk is sharpest where personal data reaches actors with strong incentives to exploit it, including data brokers and security or defence contractors that assemble personal profiles: data gathered for a mundane “writing help” feature can, once pooled and resold, support profiling and targeting well beyond its original purpose, and models can already infer sensitive attributes from ordinary text [29]. Measuring the exposure precisely, rather than assuming vendor assurances hold, is what makes the downstream risk assessable: the gap between stated safeguards and observed behaviour is exactly what this study quantifies.

VII. LIMITATIONS

We measured across several surfaces (browser extensions on an isolated Linux VM; desktop, Office, and operating-system tools on Windows 11, since several desktop tools are unavailable on Linux), which strengthens external validity. Two limits apply. First, traffic protected by certificate pinning cannot be read (for example `*.events.data.microsoft.com` and some Adobe and Google endpoints); content we cannot decrypt is not counted, so every reported figure is a lower bound. The proxy CA is trusted only in the isolated test profile, and we report the raw traffic unmodified [15]. Second, a native application can use sockets that ignore the system proxy: the Grammarly Windows client on Notepad sent only telemetry through the proxy while opening a direct connection we could not inspect, so that case is reported as unmeasured rather than zero; results captured through the proxy (browser extensions, Office, and Grammarly in Word) are unaffected.

VIII. ETHICS AND RESPONSIBLE DISCLOSURE

All test data is synthetic: no real person, organisation, or system is referenced, and no real personal data was captured or transmitted. The study measures default-configuration behaviour of legitimately installed software on the researcher’s own machine; it does not attack the vendors’ systems or other users.

IX. CONCLUSION AND FUTURE WORK

This project measured how much of a confidential document leaves the machine when common LLM-integrated productivity tools are used at their defaults. The answer is consistent: across browser extensions, a desktop client, an invoked assistant, and an ordinary editor, the tools transmit essentially the whole document, and every planted secret, to their own servers, without any step a user would recognise as “sharing.” The tools behave as designed; the finding is how far that design sits from the expectation of a user who was told not to share the file. The unifying result is that exposure is surface-dependent: content leaves the machine whenever it enters a surface an LLM-integrated service watches, not because of where the file is stored or which single application is blamed. This moves the user’s real point of control to before the text enters such a surface rather than after, and it shows the exposure is invisible, effectively complete, and repeatable rather than occasional. That one Office setting removes the largest passive leak shows the behaviour is a configuration choice, not a technical necessity.

Future work.: The clearest next direction, raised in supervision, is to classify the leaked data by type: an automatic labelling of which categories of personal information (names, contact details, financial data, identifiers) each tool transmits. This would extend the automated PII sweep with Microsoft Presidio [20], already used here to surface an unplanted session credential, into a full per-category inventory across the captures. Two mechanistic extensions also remain: dynamic instrumentation (for example Frida) to read certificate-pinned channels that are currently only a lower bound, and an

outbound-volume detector to bound opaque uploads. Further captures can raise the remaining single-run host probes to the reproducibility already achieved for the Word, DeepL, and Edge/Copilot conditions.

REFERENCES

- [1] Anupama Aggarwal, Bimal Viswanath, Liang Zhang, Saravana Kumar, Ayush Shah, and Ponnuram Kumaraguru. I spy with my little eye: Analysis and detection of spying browser extensions. In *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 47–61, 2018.
- [2] Anthropic. Detecting and countering misuse of AI: September 2026. Technical report, Anthropic, September 2026. Threat intelligence report.
- [3] Apple Inc. Apple platform security. Apple Support, 2024. iCloud data security and Advanced Data Protection. <https://support.apple.com/guide/security/welcome/web>.
- [4] Shresta Bellary Seetharam, Nabeel Mohamed, Billy Melicher, Oleksii Starov, Qing Xie, and Fang Liu. That AI extension helping you write emails? It’s reading them first. Technical report, Palo Alto Networks Unit 42, 2026. <https://unit42.paloaltonetworks.com/high-risk-gen-ai-browser-extensions/>.
- [5] Duc Bui, Brian Tang, and Kang G. Shin. Detection of inconsistencies in privacy practices of browser extensions. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P)*, pages 2780–2798, 2023. doi: 10.1109/SP46215.2023.10179338.
- [6] Nicholas Carlini, Adrienne Porter Felt, and David Wagner. An evaluation of the Google Chrome extension security architecture. In *Proceedings of the 21st USENIX Security Symposium*, 2012.
- [7] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *Proceedings of the 30th USENIX Security Symposium*, 2021.
- [8] Quan Chen and Alexandros Kapravelos. Mystique: Uncovering information leakage from browser extensions. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1687–1700, 2018.
- [9] Shuai Cheng, Haitao Xu, Shu Meng, Shuai Hao, Chuan Yue, and Zhao Li. The privacy paradox of LLMs: User perceptions and the reality of PII leakage. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, CHI ’26. ACM, 2026. doi: 10.1145/3772318.3791809.
- [10] Cloud Security Alliance. AI browser extensions: The DLP-invisible enterprise attack surface. Technical report, Cloud Security Alliance, April 2026. CSA research note, 11 April 2026. <https://labs.cloudsecurityalliance.org/research/csa-research-note-ai-browser-extension-security-gaps-2026041/>.

- [11] Andrea Continella, Yanick Fratantonio, Martina Lindorfer, Alessandro Puccetti, Ali Zand, Christopher Kruegel, and Giovanni Vigna. Obfuscation-resilient privacy leak detection for mobile apps through differential analysis. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017. doi: 10.14722/ndss.2017.23465.
- [12] Aldo Cortesi, Maximilian Hils, Thomas Kriebhbaumer, and mitmproxy contributors. Mitmproxy: A free and open source interactive HTTPS proxy, 2010–2026.
- [13] Cyberhaven Labs. 2026 AI adoption & risk report. Technical report, Cyberhaven, 2026. <https://www.cyberhaven.com/resources/report/ai-adoption-risk-report-2026>.
- [14] Savino Dambra et al. When sally met trackers: Web tracking from the users’ perspective. In *Proceedings of the 31st USENIX Security Symposium*, 2022.
- [15] Zakir Durumeric, Zane Ma, Drew Springall, Richard Barnes, Nick Sullivan, Elie Bursztein, Michael Bailey, J. Alex Halderman, and Vern Paxson. The security impact of HTTPS interception. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017.
- [16] William Enck, Peter Gilbert, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2010.
- [17] Steven Englehardt and Arvind Narayanan. Online tracking: A 1-million-site measurement and analysis. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016. doi: 10.1145/2976749.2978313.
- [18] Aurore Fass, Dolière Francis Somé, Michael Backes, and Ben Stock. DoubleX: Statically detecting vulnerable data flows in browser extensions at scale. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021.
- [19] Nils Lukas, Ahmed Salem, Robert Sim, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Analyzing leakage of personally identifiable information in language models. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P)*, pages 346–363, 2023.
- [20] Microsoft. Presidio: Data protection and de-identification SDK. <https://github.com/microsoft/presidio>, 2018. Open-source software; docs at <https://microsoft.github.io/presidio/>.
- [21] Microsoft. Data, privacy, and security for Microsoft Copilot. Microsoft Learn, 2026. Updated 2026-08-18. <https://learn.microsoft.com/en-us/microsoft-365/copilot/microsoft-365-copilot-privacy>.
- [22] Niloofar Mireshghallah, Hyunwoo Kim, Xuhui Zhou, Yulia Tsvetkov, Maarten Sap, Reza Shokri, and Yejin Choi. Can LLMs keep a secret? Testing privacy implications of language models via contextual integrity theory. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.17884.
- [23] Munir, Demir, Li, Kollnig, and Shafiq. Every keystroke you make: A tech-law measurement and analysis of event listeners for wiretapping, 2025. arXiv preprint arXiv:2508.19825.
- [24] Milad Nasr, Javier Rando, Nicholas Carlini, Jonathan Hayase, Matthew Jagielski, A. Feder Cooper, Daphne Ippolito, Christopher A. Choquette-Choo, Florian Tramèr, and Katherine Lee. Scalable extraction of training data from aligned, production language models. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. arXiv:2311.17035.
- [25] Asmit Nayak, Rishabh Khandelwal, Earlene Fernandes, and Kassem Fawaz. Experimental security analysis of sensitive data access by browser extensions. In *Proceedings of the ACM Web Conference (WWW)*, 2024.
- [26] Nikolaos Pantelaios, Nick Nikiforakis, and Alexandros Kapravelos. You’ve changed: Detecting malicious browser extensions through their update deltas. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020. doi: 10.1145/3372297.3423343.
- [27] Jingjing Ren, Ashwin Rao, Martina Lindorfer, Arnaud Legout, and David Choffnes. ReCon: Revealing and controlling PII leaks in mobile network traffic. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2016. doi: 10.1145/2906388.2906392.
- [28] Asuman Senol, Gunes Acar, Mathias Humbert, and Fredrik Zuidevereen Borgesius. Leaky forms: A study of email and password exfiltration before form submission. In *Proceedings of the 31st USENIX Security Symposium*, pages 1813–1830, 2022.
- [29] Robin Staab, Mark Vero, Mislav Balunović, and Martin Vechev. Beyond memorization: Violating privacy via inference with large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.07298.
- [30] Aleksii Starov and Nick Nikiforakis. Extended tracking powers: Measuring the privacy diffusion enabled by browser extensions. In *Proceedings of the 26th International Conference on World Wide Web (WWW)*, pages 1481–1490, 2017.
- [31] Aleksii Starov, Phillipa Gill, and Nick Nikiforakis. Are you sure you want to contact us? Quantifying the leakage of PII via website contact forms. In *Proceedings on Privacy Enhancing Technologies (PoPETs)*, pages 20–33, 2016. doi: 10.1515/popets-2015-0028.
- [32] Yash Vekaria, Aurelio Canino, Jonathan Levitsky, Alex Ciechonski, Patricia Callejo, Anna Maria Mandalari, and Zubair Shafiq. Big help or big brother? Auditing tracking, profiling and personalization in generative AI assistants. In *Proceedings of the 34th USENIX Security Symposium*, 2025.
- [33] Michael Weissbacher, Enrico Mariconti, Guillermo

Suarez-Tangil, Gianluca Stringhini, William Robertson, and Engin Kirda. Ex-ray: Detection of history-leaking browser extensions. In *Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC)*, pages 590–602, 2017.

- [34] Qinge Xie, Manoj Vignesh Kasi Murali, Paul Pearce, and Frank Li. Arcanum: Detecting and evaluating the privacy risks of browser extensions on web pages and web content. In *Proceedings of the 33rd USENIX Security Symposium*, 2024.

DECLARATION OF ORIGINALITY /

Eigenständigkeitserklärung

I hereby declare that this work is entirely my own and that I have used no sources or aids other than those indicated. All passages taken verbatim or in substance from other sources are identified as such. This work has not previously been submitted, in the same or a similar form, to any examination authority, and has not yet been published. All experimental data reported here are synthetic; no real personal data were used.

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Alle Stellen, die wörtlich oder sinngemäß aus anderen Quellen übernommen wurden, sind als solche kenntlich gemacht. Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegt und ist noch nicht veröffentlicht worden.

Stuttgart, September 17, 2026

Marawan Eldeib